Classification and Evaluation Metrics — Key Concepts

Lesson 4 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

30 October 2026 · a one-page index

Contents

The model 1
The cost function 2
Reading a classifier 2
Turning a score into a decision 2
Working with imbalance 3
More than two classes 3

An index, not a summary. Each entry says what a term means and where the handout treats it properly; the handout's own Section 11 is where the *findings* are collected. Use this to find your way back into the text, or to check before an exercise that there is no word here you could not define.

The model

Logistic regression. A linear model of the log-odds, squashed into a probability by the sigmoid. Same features and coefficients as Lesson 3, a different question. → § 2

Sigmoid. The function $\sigma(z) = 1/(1 + e^{-z})$: takes any real number and returns one between 0 and 1, steep in the middle and flat at both ends. → § 2.2

Neuron, and Rosenblatt's perceptron. Logistic regression drawn as a unit: weigh the inputs, add the intercept, squash. Rosenblatt's perceptron is the same diagram with a hard step, so it outputs a label rather than a probability; the networks of Lesson 9 are built from such units. → § 2.2

Odds and log-odds. The odds are the probability of an event divided by the probability of its complement; the log-odds are their logarithm, and they range over the whole real line, which is what a linear model can safely predict. → § 2.3

Why the log-odds, not another stretch. Other functions also map $(0, 1)$ onto the real line - the probit, built on the normal distribution, draws nearly the same curve. The log-odds is chosen because its coefficients read as odds multipliers, Bayes' rule adds evidence on its scale, and the cross-entropy cost on it is convex. Not because of the S-shape. → § 2.3

Odds multiplier. e^{w_j}: what a one-unit increase in a feature does to the odds. It multiplies the odds, not the probability — the commonest misreading in the lesson. → § 2.4

The cost function

Cross-entropy, or log loss. The cost that maximum likelihood produces for a Bernoulli outcome: the negative logarithm of the probability the model gave to what actually happened. → § 3.3

Maximum likelihood. Choose the parameters that make the observed data as unsurprising as possible. Cross-entropy is what that principle yields here rather than something anyone designed. → § 3.1

Why not squared error. Its gradient carries a factor $\sigma'(z)$ that vanishes exactly where the model is most confidently wrong, so the optimiser asks for the smallest correction where the largest is needed. → § 4.2

Convexity. The cross-entropy cost has no local minima to be trapped in — but convex does not mean solvable in closed form, and there is no normal equation for logistic regression. → § 4.3

Positive semi-definite. The property that says a surface curves upwards in every direction: $u^\top H u \geq 0$ for every u. For this cost each term of that sum is $p(1-p)$ times a square. → § 4.3

The penalty, and C. `LogisticRegression` adds lesson 3's L2 penalty to the log loss by default; its strength is $C = 1/\lambda$, so a smaller C penalises harder. Without it, classes a line separates perfectly have no finite best weights. → § 4.3

Which line the cost chooses. The boundary is the minimum of the cost, whatever solver reaches it; "best" means best by that cost. The support vector machine of Lesson 6 picks by another rule, the widest empty strip. → § 4.3

Reading a classifier

The accuracy trap. When one class is rare, answering "the majority class" every time scores well and detects nothing. This is the number to remember from the lesson. → § 5.1

Confusion matrix. The four counts — true and false, positive and negative — that every metric below is built from. Count before you summarise. → § 6.1

Precision. Of the cases the model flagged, how many really were positive. The denominator is the alarms raised. → § 6.2

Recall. Of the cases that really were positive, how many the model found. The denominator is the failures that existed. → § 6.2

F1 and F-beta. The harmonic mean of precision and recall, so a perfect score on one axis cannot buy a pass on the other; $\beta > 1$ weights recall more heavily. → § 6.3

Harmonic mean. Invert both numbers, average, invert back. Inverting is what lets the worse of the two dominate, since a small number has a large reciprocal. → § 6.3

Macro against weighted averaging. Macro gives every class an equal say; weighted gives each class its share of the data, which on an imbalanced problem means the majority class talking. → § 6.4

Turning a score into a decision

The threshold. The line between "leave it" and "act on it". Nothing about the fitted model changes as it moves — only which side of it each case falls. → § 7.1

Cost-optimal threshold. $t^* = C_{FP}/(C_{FP} + C_{FN})$. For calibrated probabilities, with each cost counted as what the error adds over the right decision, a function of the two error costs alone: not of the model, the dataset, or the class balance. → § 7.2

Ranking. The cases sorted by predicted probability, riskiest first. Every threshold is a cut in this list, so a measure of the ranking judges every threshold at once. → § 8.1

Receiver operating characteristic (ROC) curve. True positive rate against false positive rate, swept over every threshold. Its shape says how fast the model catches positives relative to how fast it raises false alarms. → § 8.1

Area under the curve (AUC). The probability that the model scores a randomly chosen positive above a randomly chosen negative. It measures ranking, and is blind to whether the probabilities themselves mean anything. → § 8.2

Calibration. Whether the probabilities mean what they say: among cases given 0.7, about 70% are positive. Independent of ranking: a model can rank well and be badly calibrated, or be calibrated and rank nothing. → § 7.2, § 8.2

Average precision (AP). The area under the precision-recall curve, computed as a step sum: the precision at each threshold, weighted by the recall that threshold adds (not a trapezoid drawn through the points). A random ranking scores the positive rate, so AP is read against that, not 0.5. → § 7.1

Where AUC misleads. It is unmoved by class imbalance, because its denominator is the whole negative class; precision is not. On a rare-event problem report the precision-recall curve beside it. → § 8.3

Working with imbalance

Class weights. Make each rare example count as several during training. Note that this moves the operating point rather than teaching the model anything new — close to the threshold by another name: weights in the ratio k act like a threshold of $1/(1 + k)$, exactly only for a correctly specified model and a large sample. "Balanced" sets the ratio from the class frequencies, not from the costs. → § 9

Resampling. Oversampling the minority class, or synthesising new minority examples. Same caution, plus one more: it must happen inside the cross-validation fold. → § 9

More than two classes

Softmax. One score per class, exponentiated and normalised to a probability distribution. For two classes it reduces to the sigmoid. → § 10.1

Categorical cross-entropy. The same idea as the binary loss, generalised: the negative logarithm of the probability assigned to the true class. → § 10.1

One-against-rest metrics. Precision and recall computed one class at a time, each in turn treated as the positive one. → § 10.2